

Bài thực hành MongoDB: Hệ thống quản lý bán hàng (Sales Management)

0. Mục tiêu

Sau khi hoàn thành bài thực hành, sinh viên có khả năng:

- Thực hiện đầy đủ bốn thao tác CRUD (Create, Read, Update, Delete) trên MongoDB.
- Đọc hiểu một thiết kế schema document đã chuẩn bị sẵn, giải thích được lý do từng phần dữ liệu được nhúng (embedding) hoặc tách ra liên kết (referencing).
- Tạo index đơn (single-field) và index cụm (compound index), sử dụng `explain()` để kiểm chứng index có được sử dụng hay không.
- Viết aggregation pipeline nhiều bước (`$match`, `$group`, `$unwind`, `$lookup`, `$sort`, `$project`) để tính báo cáo doanh thu, khách hàng và sản phẩm.
- Tự viết một đoạn script (ngôn ngữ tự chọn) để bổ sung dữ liệu vào MongoDB, không chỉ chạy lại lệnh đã cho sẵn.
- Thao tác được với MongoDB bằng cả dòng lệnh và công cụ đồ hoạ (MongoDB Compass).

Bài thực hành sử dụng chủ đề quản lý bán hàng, độc lập với bộ dữ liệu demo đã trình bày trong buổi giảng (khác từ tên collection đến cách phối hợp embedding/linking). Mục đích là để sinh viên tự thao tác lại từ đầu, không chỉ chạy lại đúng script đã xem trên lớp.

Bài thực hành gồm năm bài nhỏ (Bài 1 đến Bài 5). Cả năm bài đều bắt buộc thực hiện, không có bài nào là tùy chọn.

0.1. Quy ước trình bày trong tài liệu

Mỗi phần trong năm bài dưới đây thuộc một trong hai dạng, được ghi chú rõ:

Ví dụ mẫu. Phần có sẵn lệnh và code đầy đủ, mục đích minh hoạ cú pháp. Nên chạy thử và đọc hiểu trước khi chuyển sang phần "Tự làm" ngay sau đó.

Tự làm. Phần chỉ mô tả yêu cầu cần đạt (đầu vào, kết quả mong đợi), không kèm sẵn lệnh hay code. Sinh viên được phép sử dụng công cụ AI (ChatGPT, Claude, Copilot...) hỗ trợ viết, nhưng khi nộp bài phải tự giải thích được từng phần chính của lệnh hoặc đoạn code đã viết. Một lệnh dán vào mà không giải thích được sẽ xem như chưa hoàn thành phần đó.

1. Điều kiện trước khi bắt đầu

Bài thực hành giả định sinh viên đã biết cách khởi động một MongoDB, nội dung này đã học ở buổi giảng MongoDB. Tài liệu không nhắc lại việc cài đặt.

Bài thực hành không bắt buộc phải dùng đúng container Docker đi kèm. Điều kiện duy nhất là có một MongoDB đang chạy (bản `mongo:8.0` trở lên) mà sinh viên biết địa chỉ kết nối (connection string, dạng `mongodb://host:port`). Nguồn MongoDB có thể là:

- container Docker dựng từ `docker-compose.yml` trong thư mục này, là cách nhanh nhất nếu chưa có sẵn MongoDB nào khác, connection string `mongodb://localhost:27017`;

- MongoDB đã cài trực tiếp trên máy, không qua Docker;
- MongoDB Atlas bản miễn phí, cluster chạy trên cloud, chỉ cần connection string do Atlas cấp.

Toàn bộ lệnh trong tài liệu này giả định dùng `mongodb://localhost:27017` (container Docker đi kèm). Nếu dùng nguồn khác, chỉ cần thay connection string tương ứng, các lệnh còn lại giữ nguyên.

2. Hai cách thao tác: dòng lệnh (mongosh) hoặc giao diện đồ họa (Compass)

Tài liệu trình bày lệnh dưới dạng `mongosh` vì ngắn gọn và dễ chép lại vào bài nộp, nhưng mọi thao tác đều có thể thực hiện tương đương bằng MongoDB Compass, công cụ GUI chính thức của MongoDB, tải miễn phí tại trang chủ MongoDB. Sinh viên chọn cách quen thuộc hơn, có thể kết hợp cả hai.

Việc cần làm	Bằng dòng lệnh (mongosh)	Bằng Compass
Kết nối	<code>mongosh</code> <code>"mongodb://localhost:27017"</code>	Mở Compass, dán connection string <code>mongodb://localhost:27017</code> , chọn Connect
Nạp file JSON	<code>mongoimport ... --jsonArray</code>	Chọn database/collection, chọn ADD DATA rồi Import File, chọn file <code>.json</code> , định dạng JSON
Xem và lọc document	<code>db.orders.find({...})</code>	Tab Documents, gõ điều kiện lọc vào ô Filter theo đúng cú pháp JSON như <code>find()</code>
Aggregation pipeline	<code>db.orders.aggregate([...])</code>	Tab Aggregations, thêm từng stage bằng giao diện, hoặc dán nguyên mảng stage ở chế độ "as text"
Tạo index	<code>db.orders.createIndex({...})</code>	Tab Indexes, chọn Create Index, chọn trường và chiều tăng/giảm
Xem <code>explain()</code>	<code>.explain("executionStats")</code>	Trong tab Documents/Aggregations có nút Explain Plan cạnh ô Filter
Chạy lệnh JS tự do	<code>mongosh</code> tương tác	Compass có sẵn tab MONGOSH ở góc dưới bên trái cửa sổ, một <code>mongosh</code> nhúng ngay trong Compass, chạy được đúng các lệnh JS trong tài liệu này

Ảnh chụp màn hình nộp bài có thể lấy từ terminal `mongosh` hoặc từ cửa sổ Compass, cả hai đều hợp lệ, miễn thấy rõ lệnh hoặc thao tác và kết quả.

4. Chủ đề bài toán

Một cửa hàng bán lẻ cần quản lý bốn khối dữ liệu có quan hệ với nhau: khách hàng mua sắm, sản phẩm đang bán, đơn hàng khách đặt (mỗi đơn có nhiều dòng sản phẩm), và đánh giá sản phẩm do khách hàng để lại sau khi mua. Bốn khối dữ liệu này được mô hình hoá theo hai cách khác nhau trong MongoDB, là nội dung chính của phần thiết kế schema ở mục 5, sinh viên cần đọc hiểu trước khi thao tác.

5. Thiết kế schema

MongoDB không ép buộc schema cố định như bảng trong CSDL quan hệ, nhưng một hệ thống thực tế vẫn cần thiết kế trước cấu trúc document cho từng collection, để thống nhất cách đọc và ghi dữ liệu giữa các thành phần trong hệ thống. Phần dưới đây trình bày schema đã thiết kế sẵn cho bốn collection, kèm giải thích từng trường và lý do chọn embedding hay referencing cho từng collection.

5.0. Về trường `_id` và các trường id ở tầng ứng dụng

Mỗi document có hai loại định danh khác nhau, cần phân biệt rõ:

`_id` là khoá chính do chính MongoDB tự sinh, kiểu ObjectId, dài 12 byte, không có sẵn trong file JSON nguồn, được tự động gán khi thực hiện `insertOne` hoặc `mongoimport`.

`customer_id`, `product_id`, `order_id`, `review_id` là id do tầng ứng dụng (ng nghiệp vụ) tự đặt ra và quản lý, dùng để tra cứu và liên kết giữa các collection.

Trong hệ thống thực tế, id ở tầng ứng dụng thường được sinh dưới dạng GUID hoặc UUID (chuỗi ngẫu nhiên, ví dụ `550e8400-e29b-41d4-a716-446655440000`) để đảm bảo không trùng lặp khi nhiều thành phần cùng sinh dữ liệu độc lập. Bộ dữ liệu của bài thực hành này chủ động dùng chuỗi id ngắn và dễ đọc, ví dụ `CUST0001`, `PROD0001`, thay vì GUID, nhằm thuận tiện quan sát và đối chiếu khi học. Về bản chất, các trường này vẫn là cùng một khái niệm: id ở tầng ứng dụng. Tài liệu gọi thẳng các trường này là "id", không dùng từ "mã" để tránh gây nhầm lẫn với khái niệm "mã nguồn" (code) khi trao đổi bằng lời.

5.1. Collection `customers`

Collection độc lập, không nhúng thêm dữ liệu từ collection khác.

Trường	Kiểu dữ liệu	Giải thích
<code>_id</code>	ObjectId	Khoá chính do MongoDB tự sinh khi <code>insertOne</code> hoặc <code>mongoimport</code> .
<code>customer_id</code>	string	Id khách hàng ở tầng ứng dụng, dùng làm khoá tra cứu và liên kết (" <code>CUST0001</code> "), khác <code>_id</code> (xem mục 5.0).
<code>full_name</code>	string	Họ tên khách hàng.
<code>email, phone</code>	string	Thông tin liên hệ.
<code>customer_segment</code>	string (enum)	Phân khúc khách hàng: <code>NEW</code> , <code>REGULAR</code> , <code>VIP</code> , <code>CHURN_RISK</code> (có nguy cơ rời bỏ).
<code>city</code>	string	Thành phố.
<code>address</code>	string	Địa chỉ giao hàng mặc định.
<code>created_at</code>	string (ISO 8601)	Thời điểm tạo tài khoản.

5.2. Collection `products`

Collection độc lập, có một sub-document được nhúng bên trong.

Trường	Kiểu dữ liệu	Giải thích
<code>_id</code>	ObjectId	Tự sinh.
<code>product_id</code>	string	Id sản phẩm ở tầng ứng dụng ("PROD0001").
<code>category_id</code> , <code>category_name</code>	string	Danh mục sản phẩm.
<code>product_name</code> , <code>brand</code>	string	Tên sản phẩm, thương hiệu.
<code>price</code>	number	Đơn giá, tính bằng VND.
<code>stock_quantity</code>	number	Số lượng tồn kho.
<code>attributes</code>	object nhúng	<code>{ color, warranty_months }</code> . Đây là thuộc tính mô tả sản phẩm, luôn đi kèm sản phẩm và không bao giờ cần truy vấn độc lập, nên được nhúng thẳng thay vì tách thành collection riêng.
<code>tags</code>	array<string>	Nhãn phân loại thêm, ví dụ <code>ban_chay</code> , <code>moi_ve</code> , dùng cho lọc và tìm kiếm.
<code>created_at</code>	string (ISO 8601)	Thời điểm thêm sản phẩm.

5.3. Collection `orders`

Collection minh hoạ rõ nhất cho embedding, có hai lớp dữ liệu được nhúng.

Trường	Kiểu dữ liệu	Giải thích
<code>_id</code>	ObjectId	Tự sinh.
<code>order_id</code>	string	Id đơn hàng ("ORD00001").
<code>customer</code>	object nhúng	Bản sao rút gọn thông tin khách hàng tại thời điểm đặt đơn: <code>{ customer_id, full_name, city, customer_segment }</code> . Thông tin này được nhúng vì khi xem một đơn hàng, hệ thống hầu như luôn cần hiển thị ngay tên và khu vực khách hàng, việc nhúng giúp đọc một lần là đủ, không cần <code>\$lookup</code> sang <code>customers</code> . Đánh đổi là nếu khách đổi tên hoặc địa chỉ sau này, các đơn hàng cũ vẫn giữ thông tin cũ tại thời điểm mua. Đây là hành vi có chủ đích, không phải lỗi, tương tự cách một hoá đơn giấy ghi lại đúng thông tin tại thời điểm giao dịch.
<code>order_time</code>	string (ISO 8601)	Thời điểm đặt đơn.
<code>status</code>	string (enum)	<code>CREATED</code> , <code>PAID</code> , <code>SHIPPED</code> , <code>CANCELLED</code> .

Trường	Kiểu dữ liệu	Giải thích
<code>payment_method</code>	string (enum)	<code>COD</code> , <code>BANK_TRANSFER</code> , <code>E_WALLET</code> , <code>CREDIT_CARD</code> .
<code>shipping_address</code>	string	Địa chỉ giao hàng của đơn này.
<code>items</code>	array nhúng	Danh sách dòng sản phẩm trong đơn, mỗi phần tử gồm { <code>product_id</code> , <code>product_name</code> , <code>unit_price</code> , <code>quantity</code> , <code>line_total</code> }. Dữ liệu này được nhúng vì các dòng sản phẩm của một đơn hàng luôn được đọc và ghi cùng lúc với chính đơn hàng đó, không ai truy vấn một dòng sản phẩm tách rời khỏi đơn hàng, và số lượng dòng thường nhỏ (tối đa vài chục), an toàn với giới hạn 16MB cho mỗi document.
<code>total_amount</code>	number	Tổng tiền đơn hàng, bằng tổng <code>line_total</code> của <code>items</code> .
<code>note</code>	string	Ghi chú, có thể để trống.

5.4. Collection `reviews`

Collection minh hoạ cho referencng, liên kết tới `products` và `customers` thay vì nhúng trực tiếp.

Trường	Kiểu dữ liệu	Giải thích
<code>_id</code>	ObjectId	Tự sinh.
<code>review_id</code>	string	Id đánh giá ("REV00001").
<code>product_id</code>	string	Không nhúng toàn bộ document sản phẩm, chỉ lưu id liên kết, tương tự khoá ngoại trong CSDL quan hệ.
<code>customer_id</code>	string	Tương tự, chỉ lưu id liên kết tới <code>customers</code> .
<code>rating</code>	number (1 đến 5)	Điểm đánh giá.
<code>comment</code>	string	Nội dung nhận xét.
<code>created_at</code>	string (ISO 8601)	Thời điểm đánh giá.

Lý do `reviews` dùng referencng thay vì embedding: một sản phẩm có thể có rất nhiều đánh giá theo thời gian, không giới hạn số lượng như `items` trong `orders`, và đánh giá cần được truy vấn độc lập với sản phẩm. Ví dụ, truy vấn "lấy mười đánh giá mới nhất của một khách hàng trên toàn hệ thống" không thể thực hiện được nếu đánh giá bị nhúng cứng bên trong từng document `products`. Đây là ví dụ đối lập trực tiếp với `orders.items` ở mục 5.3, giúp sinh viên so sánh hai tình huống nên nhúng và nên tách.

6. Khởi động MongoDB và nạp dữ liệu

Chọn một trong ba cách dưới đây tùy hoàn cảnh, kết quả cuối cùng giống nhau: một database `salesmgmt` với bốn collection đã nạp dữ liệu.

Cách A: Docker (đi kèm sẵn, nhanh nhất nếu chưa có MongoDB nào khác)

```
cd Labs/01_mongodb
docker compose up -d
docker exec mongodb_lab mongosh --quiet --eval "db.version()"
```

Kết quả mong đợi: 8.0.29.

```
export MSYS_NO_PATHCONV=1 # chỉ cần trên Git Bash Windows, PowerShell thì bỏ qua dòng này

docker cp data/customers.json mongodb_lab:/tmp/customers.json
docker cp data/products.json mongodb_lab:/tmp/products.json
docker cp data/orders.json mongodb_lab:/tmp/orders.json
docker cp data/reviews.json mongodb_lab:/tmp/reviews.json

docker exec mongodb_lab mongoimport --db salesmgmt --collection customers --file /tmp/customers.json --jsonArray
docker exec mongodb_lab mongoimport --db salesmgmt --collection products --file /tmp/products.json --jsonArray
docker exec mongodb_lab mongoimport --db salesmgmt --collection orders --file /tmp/orders.json --jsonArray
docker exec mongodb_lab mongoimport --db salesmgmt --collection reviews --file /tmp/reviews.json --jsonArray
```

Cách B: MongoDB cài trực tiếp trên máy (có sẵn mongosh và mongoimport trong PATH)

Thực hiện tương tự Cách A nhưng bỏ tiền tố `docker exec mongodb_lab` và `docker cp`, trở thành `mongoimport` vào file trong `data/`:

```
mongoimport --db salesmgmt --collection customers --file data/customers.json --jsonArray
mongoimport --db salesmgmt --collection products --file data/products.json --jsonArray
mongoimport --db salesmgmt --collection orders --file data/orders.json --jsonArray
mongoimport --db salesmgmt --collection reviews --file data/reviews.json --jsonArray
```

Cách C: MongoDB Compass (không cần dòng lệnh)

1. Mở Compass, kết nối bằng connection string tương ứng: `mongodb://localhost:27017` nếu dùng Cách A hoặc B, hoặc chuỗi kết nối do Atlas cấp nếu dùng cloud.
2. Tạo database `salesmgmt`, tạo lần lượt bốn collection: `customers`, `products`, `orders`, `reviews`.
3. Vào từng collection, chọn ADD DATA rồi Import File, chọn đúng file JSON tương ứng trong `data/`, chọn định dạng JSON, thực hiện Import.

Kết quả mong đợi ở cả ba cách: **100 / 80 / 250 / 200** document tương ứng ở **customers / products / orders / reviews**, không có lỗi. Nếu import báo lỗi, ví dụ liên quan đến **_id** hoặc ObjectId, chụp lại lỗi đó, đây cũng là minh chứng cần nộp.

Kiểm tra nhanh sau khi nạp dữ liệu:

```
docker exec mongodb_lab mongosh salesmgmt --quiet --eval "  
print('customers:', db.customers.countDocuments());  
print('products:', db.products.countDocuments());  
print('orders:', db.orders.countDocuments());  
print('reviews:', db.reviews.countDocuments());  
"
```

Thay `docker exec mongodb_lab mongosh salesmgmt --quiet --eval "..."` bằng `mongosh salesmgmt --quiet --eval "..."` nếu dùng Cách B, hoặc dùng tab Documents của Compass để xem số lượng document nếu dùng Cách C.

Từ đây, có thể mở phiên **mongosh** tương tác để thực hiện các bài bên dưới:

```
docker exec -it mongodb_lab mongosh salesmgmt
```

Bài 1. CRUD cơ bản

Ví dụ mẫu: Create và Read trên **products**

```
db.products.insertOne({  
  product_id: "PROD9001",  
  category_id: "CAT01",  
  category_name: "Dien tu",  
  product_name: "Dien tu Test #9001",  
  brand: "Aurora",  
  price: 5990000,  
  stock_quantity: 20,  
  attributes: { color: "den", warranty_months: 12 },  
  tags: ["moi_ve"],  
  created_at: new Date().toISOString(),  
});  
  
db.products.findOne({ product_id: "PROD9001" });
```

Trong hai lệnh trên, **insertOne** nhận vào đúng một object theo schema ở mục 5.2. **findOne** tìm lại bằng điều kiện lọc, ở đây là **product_id** do tự đặt, không phải **_id** do MongoDB tự sinh.

Tự làm: Update và Delete trên **products** (bắt buộc)

Với đúng sản phẩm **PROD9001** vừa tạo ở trên, tự viết lệnh cho hai yêu cầu sau.

1. Update: giảm giá sản phẩm đó 10%, tự tính giá mới, dùng `updateOne` với toán tử `$set`. Đọc kết quả trả về, giải thích ý nghĩa `matchedCount` và `modifiedCount`.
2. Delete: xóa sản phẩm đó bằng `deleteOne`. Đọc `deletedCount`, sau đó gọi lại `findOne` để xác nhận đã xóa thành công, kết quả phải là `null`.

Tự làm: lặp lại đủ bốn thao tác trên `customers` (bắt buộc)

Tự chọn dữ liệu, tự đặt `customer_id` mới theo đúng schema ở mục 5.1, tự viết cả bốn lệnh Create, Read, Update, Delete từ đầu. Phần này không có sẵn mẫu.

Bài 2. Đánh index và kiểm chứng bằng `explain()`

Ví dụ mẫu: index đơn (single-field) trên `customers`

Một truy vấn phổ biến trên `customers` là tìm theo `customer_segment`, ví dụ lọc riêng nhóm `VIP` để chạy chiến dịch marketing.

```
db.customers.createIndex({ customer_segment: 1 });
db.customers.find({ customer_segment: "VIP" }).explain("executionStats");
```

Trong kết quả `explain()`, cần đọc hai giá trị quan trọng: `winningPlan.stage` (nếu là `COLLSCAN` nghĩa là quét toàn bộ collection, nếu là `FETCH` với `inputStage.stage = IXSCAN` nghĩa là đã dùng index) và `executionStats.totalDocsExamined` so với `nReturned`.

Tự làm: index cụm (compound index) trên `orders` (bắt buộc)

Yêu cầu nghiệp vụ: tìm nhanh các đơn đã thanh toán (`status = "PAID"`), sắp xếp theo thời gian đặt hàng mới nhất trước. Một truy vấn vừa lọc vừa sắp xếp như vậy nên dùng một index cụm gồm hai trường, thay vì hai index đơn riêng biệt.

Tự viết lệnh tạo index cụm phù hợp trên `orders`, lệnh truy vấn tương ứng (`find` kết hợp `sort`), và chạy `explain("executionStats")` để kiểm chứng.

Câu hỏi cần trả lời trong bài nộp: vì sao index cụm vừa tạo phục vụ được cả điều kiện lọc lẫn điều kiện sắp xếp chỉ trong một lần quét.

2.3. Trường hợp index chưa thể hiện rõ tác dụng

Với chỉ vài trăm document, MongoDB đôi khi vẫn chọn quét toàn bộ collection (`COLLSCAN`) thay vì dùng index, vì dữ liệu quá ít nên chi phí không khác nhau đáng kể. Đây là hành vi bình thường, không phải lỗi. Nếu ở phần trên vẫn thấy `COLLSCAN`, chuyển sang Bài 5 để tự thêm dữ liệu và quan sát lại.

Bài 3. Aggregation pipeline

Ví dụ mẫu: doanh thu theo danh mục sản phẩm

Chỉ tính các đơn đã thanh toán hoặc đã giao.

```

db.orders.aggregate([
  { $match: { status: { $in: ["PAID", "SHIPPED"] } } },
  { $unwind: "$items" },
  {
    $lookup: {
      from: "products",
      localField: "items.product_id",
      foreignField: "product_id",
      as: "product",
    },
  },
  { $unwind: "$product" },
  {
    $group: {
      _id: "$product.category_name",
      revenue: { $sum: "$items.line_total" },
      order_line_count: { $sum: 1 },
    },
  },
  { $sort: { revenue: -1 } },
]);

```

Trong pipeline này, `$match` lọc trước để giảm dữ liệu càng sớm càng tốt, `$unwind` tách mảng `items` thành nhiều document con, `$lookup` nối sang `products` để lấy `category_name`, `$group` gộp và tính tổng theo danh mục.

Tự làm: top 5 khách hàng chi tiêu nhiều nhất (bắt buộc)

Yêu cầu: từ collection `orders`, chỉ tính các đơn `PAID` hoặc `SHIPPED`, nhóm theo `customer.customer_id`, tính tổng `total_amount` và số đơn hàng, lấy ra năm khách hàng có tổng chi tiêu cao nhất, kèm tên khách hàng trong kết quả.

Tự viết pipeline. Gợi ý các stage cần dùng: `$match`, `$group` (dùng `$first` để giữ lại tên khách hàng), `$sort`, `$limit`. Phần này không có sẵn code.

Tự làm: điểm đánh giá trung bình mỗi sản phẩm (bắt buộc)

Yêu cầu: từ collection `reviews`, tính điểm `rating` trung bình và số lượng đánh giá theo từng `product_id`, sau đó nối sang `products` để lấy `product_name`. Đây là pipeline dùng đúng cặp collection referencing ở mục 5.4, bắt buộc phải `$lookup` vì `reviews` không nhúng sẵn tên sản phẩm. Lấy ra năm sản phẩm có điểm trung bình cao nhất.

Tự viết pipeline. Gợi ý các stage cần dùng: `$group`, `$lookup`, `$unwind`, `$sort`, `$limit`. Có thể tham khảo cách `$lookup` được dùng ở ví dụ mẫu đầu bài để nắm cú pháp, nhưng phải tự ghép lại đúng bài toán này. Phần này không có sẵn code.

Bài 4. So sánh embedding và referencing bằng thao tác thật

Toàn bộ bài này thuộc dạng tự làm, không có sẵn code.

1. Lấy một đơn hàng bất kỳ trên `orders`, dùng `findOne` hoặc tab Documents của Compass. Quan sát mảng `items` đã có sẵn ngay trong kết quả, không cần thêm bước nào khác.
2. Lấy toàn bộ đánh giá của một sản phẩm bất kỳ trên `reviews`, lọc theo `product_id`. Quan sát kết quả không có tên sản phẩm, phải dùng `$lookup` như ở Bài 3 mới ghép được.
3. Viết ba đến năm câu, bằng lời văn của sinh viên, không sao chép nguyên mục 5.3 hoặc 5.4, giải thích: nếu đổi `reviews` sang embedding thẳng vào `products`, mỗi sản phẩm tự chứa mảng review của chính nó, điều gì có thể xảy ra khi một sản phẩm bán chạy có hàng nghìn đánh giá.

Bài 5. Tự viết script thêm dữ liệu để thấy tác dụng thật của index (bắt buộc)

Với chỉ vài trăm document, index ở Bài 2 có thể chưa cho thấy khác biệt rõ so với quét toàn bộ collection. Bài này yêu cầu tự viết một script, không dùng lệnh có sẵn trong tài liệu vì đây chính là phần thực hành viết code chứ không phải chép lại, để thêm một lượng lớn document vào collection `orders`, đủ để index cụm ở Bài 2 thể hiện rõ tác dụng.

Yêu cầu cụ thể:

- Ngôn ngữ và công cụ tự chọn, ví dụ Python với thư viện `pymongo`, Node.js với thư viện `mongodb`, một file `.js` chạy bằng `mongosh script.js`, hoặc bất kỳ cách nào khác miễn kết nối được tới MongoDB đang dùng.
- Thêm tối thiểu khoảng 20.000 đến 50.000 document mới vào `orders`.
- Document mới phải đúng theo schema đã thiết kế ở mục 5.3, đầy đủ các trường `order_id`, `customer`, `order_time`, `status`, `items`, `total_amount`. Các trường `status` và `order_time` phải đa dạng, nhiều giá trị `status` khác nhau, `order_time` trải rộng trên một khoảng thời gian. Nếu toàn bộ document mới giống hệt nhau ở đúng trường mà index đang dùng, index sẽ không có gì để phân biệt.
- Được phép dùng AI hỗ trợ viết script, nhưng phải tự giải thích được cách script kết nối tới MongoDB, logic vòng lặp sinh dữ liệu hoạt động thế nào, và vì sao chọn cách thêm dữ liệu theo lô (batch insert) thay vì thêm từng document một khi số lượng lớn.

Sau khi thêm dữ liệu, chạy lại đúng lệnh `explain()` ở Bài 2, phần index cụm, để so sánh `totalDocsExamined`, `executionTimeMillis` và `winningPlan.stage` trước và sau khi có thêm dữ liệu. Đây là lúc index cụm mới thể hiện rõ tác dụng, chỉ quét đúng số document khớp điều kiện thay vì toàn bộ collection.

Không bắt buộc dọn lại dữ liệu vừa thêm sau khi làm xong Bài 5. Nếu chủ động đặt tiền tố riêng cho `order_id` khi tự sinh dữ liệu, ví dụ `"PERFTEST..."`, có thể tự viết thêm một lệnh `deleteMany` để dọn sạch theo điều kiện đó.

7. Dừng và dọn dẹp

```
docker compose down
```

Giữ thư mục `data_db/` nếu muốn giữ lại dữ liệu đã nạp cho lần chạy sau. Xóa thư mục này nếu muốn chạy lại từ đầu, bằng cách `docker compose down` rồi `rm -rf data_db` trước khi `docker compose up -d` lại.

8. Yêu cầu nộp bài

Nộp lại một file Word (.docx) gồm các nội dung sau.

- 1. Với mỗi bài, từ Bài 1 đến Bài 5, tất cả đều bắt buộc: chép nguyên văn từng lệnh hoặc đoạn code đã chạy, kèm ảnh chụp màn hình kết quả trả về ngay dưới lệnh đó. Ảnh chụp từ terminal `mongosh` hoặc từ Compass đều hợp lệ.
- 2. Với các phần được đánh dấu "Tự làm": ngoài lệnh và ảnh chụp kết quả, viết thêm hai đến bốn câu giải thích từng phần chính của lệnh hoặc đoạn code đó làm gì. Không cần giải thích lý thuyết dài dòng, chỉ cần đủ cho thấy đã tự hiểu, kể cả khi có dùng AI hỗ trợ viết.
- 3. Với Bài 2, phần index cụm: bắt buộc có ảnh chụp `explain()` trước và sau khi tạo index, đối chiếu `winningPlan.stage`.
- 4. Với Bài 4, mục 3: đoạn văn tự viết, không sao chép từ tài liệu này.
- 5. Với Bài 5: nêu rõ ngôn ngữ và công cụ đã chọn, dán toàn bộ script đã viết, kèm ảnh chụp `explain()` trước và sau khi thêm dữ liệu.
- 6. Ghi rõ họ tên và mã sinh viên ở đầu file.

Không cần nộp lại toàn bộ log, chỉ cần đủ minh chứng đã tự tay chạy và hiểu đúng từng bước.

Phụ lục: bảng thuật ngữ

Thuật ngữ	Giải thích
Document	Một bản ghi trong MongoDB, tương tự một dòng trong bảng Excel nhưng linh hoạt hơn.
Collection	Một nhóm document, tương ứng khái niệm bảng trong CSDL quan hệ.
CRUD	Bốn thao tác cơ bản: Create (tạo), Read (đọc), Update (sửa), Delete (xoá).
Embedding	Gộp dữ liệu liên quan vào chung một document, ví dụ nhúng mảng <code>items</code> vào chính <code>orders</code> . Đọc nhanh do chỉ cần một lần truy vấn, nhưng document có thể phình to nếu dữ liệu con không giới hạn.
Referencing (linking)	Tách dữ liệu liên quan ra collection riêng, chỉ lưu id liên kết, tương tự khoá ngoại. Linh hoạt hơn khi dữ liệu con lớn hoặc không giới hạn, nhưng khi đọc phải dùng <code>\$lookup</code> để ghép lại.
Index	Cấu trúc dữ liệu phụ giúp tìm kiếm nhanh hơn thay vì quét toàn bộ collection (<code>COLLSCAN</code>). Index đơn dùng một trường, index cụm (compound) dùng nhiều trường theo đúng thứ tự khai báo.
<code>explain()</code>	Lệnh yêu cầu MongoDB cho biết vừa tìm kiếm bằng cách nào, dùng để kiểm tra index có thực sự được optimizer sử dụng hay không (<code>IXSCAN</code>) hay vẫn quét toàn bộ (<code>COLLSCAN</code>).
Aggregation Pipeline	Chuỗi các bước xử lý dữ liệu nối tiếp nhau, ví dụ <code>\$match</code> để lọc, <code>\$lookup</code> để nối, <code>\$unwind</code> để tách mảng, <code>\$group</code> để gộp và tính tổng, <code>\$sort</code> để sắp xếp, nhằm tạo ra báo cáo tổng hợp. Tương tự <code>GROUP BY</code> nhiều bước trong SQL.
<code>\$lookup</code>	Bước aggregation nối dữ liệu giữa hai collection theo một trường khoá, tương tự <code>JOIN</code> trong SQL.

Thuật ngữ	Giải thích
<code>\$unwind</code>	Bước aggregation tách một mảng thành nhiều document riêng, mỗi phần tử mảng thành một document, thường dùng trước khi <code>\$group</code> theo từng phần tử.
ObjectId	Kiểu dữ liệu MongoDB tự sinh cho trường <code>_id</code> nếu không tự cung cấp, dài 12 byte, đảm bảo gần như không trùng lặp giữa các document.
GUID (UUID)	Chuỗi định danh ngẫu nhiên, ví dụ <code>550e8400-e29b-41d4-a716-446655440000</code> , thường dùng làm id tăng ứng dụng trong hệ thống thực tế để tránh trùng lặp khi sinh phân tán. Bài thực hành này dùng id dạng chuỗi để đọc thay vì GUID, xem mục 5.0.
MongoDB Compass	Công cụ giao diện đồ hoạ chính thức của MongoDB, cho phép xem và lọc document, xây dựng aggregation pipeline, tạo index, xem <code>explain()</code> , tất cả bằng thao tác chuột thay vì gõ lệnh.